

School of Future Tech

Mini Project – Report
on

Smart Learning System

Mini Project — React

Problem Statement 05 | Group 5

by

Subrata Panda

Japji Kaur

Sanika Kangane

Parth Sarova

Cohort: Larry Page

Academic Year 2025–2029

Report

React | Vite | Tailwind CSS | React Router | Recharts

Index

No.	Section	Page
1	Abstract	3
2	Introduction	3
3	Objectives	4
4	Problem Statement	4
5	Core Features	4
6	Tech Stack & Methods Applied	5
7	Project Architecture	6
8	File Structure	7
9	How to Run the Project	8
10	Key Implementation Details	9
11	Website Screenshots	10
12	Time & Space Complexity	11
13	Results and Conclusion	11
14	Future Enhancements and References	12

1. Abstract

SmartLearn is a full-stack web application developed as a mini-project using modern front-end technologies including React, Vite, Tailwind CSS, React Router, and Recharts. The platform is designed to simulate a complete Smart Learning System that provides students with structured course access, video lesson playback, interactive quizzes with real-time scoring, progress tracking dashboards, a community discussion forum, downloadable study resources, course completion certificates, and instructor profiles.

The project implements component-based architecture, client-side routing, mock data management, and responsive UI design to replicate a production-grade educational platform. A public-facing landing page and an authenticated dashboard layout separate the marketing presence from the application interface, closely mirroring industry-standard SaaS product design.

2. Introduction

Online education platforms have become critical infrastructure for modern learners. Students require organized course catalogs, on-demand video lessons, progress monitoring, peer discussion channels, and credential verification. Building such a system from scratch demonstrates the integration of multiple front-end technologies and design patterns used in real-world software products.

SmartLearn was developed to demonstrate how React's component model, combined with Tailwind CSS utility-first styling, React Router navigation, and Recharts data visualization, can produce a cohesive and fully functional learning management system. The project uses a centralized mock data layer to simulate backend responses, making it a complete standalone front-end application with no external API dependencies.

The design philosophy of SmartLearn follows a black-and-white minimal aesthetic inspired by productivity tools such as Notion, ensuring high readability, professional appearance, and accessibility across all screen sizes.

3. Objectives

- To build a production-grade learning platform using React and Vite.
- To implement a multi-page application using React Router with nested routes.
- To create a landing page that converts visitors into platform users.
- To develop a course catalog with filtering, searching, and progress tracking.
- To build an interactive video player simulation with lesson navigation.
- To implement a fully functional quiz engine with scoring and result display.
- To visualize learning analytics using Recharts bar and line charts.
- To provide a discussion forum with threaded replies and instructor badges.
- To generate downloadable study resources organized by course and type.
- To produce printable course completion certificates with unique identifiers.
- To showcase instructor profiles with modal pop-up detail views.
- To demonstrate reusable component design patterns across the application.

4. Problem Statement

Students learning through self-directed online platforms often struggle with fragmented tools — videos on one platform, notes on another, quizzes elsewhere. A unified system is needed that brings course content, progress tracking, peer discussion, and certification together in one seamless interface. The system must be responsive,

visually clear, and functional without requiring a live backend, making it suitable for academic demonstration and prototyping.

5. Core Features

Feature	Description
Landing Page	Public-facing marketing page with hero, features, pricing, testimonials, and CTA sections.
Course Catalog	Filterable and searchable grid of all available courses with category and level filters.
Course Detail Page	Full course overview with tabbed sections for Overview, Lessons, Quizzes, and Resources.
Video Player	Simulated video player with playback controls, progress bar scrubbing, and lesson navigation.
Interactive Quiz	5-question quizzes with instant A/B/C/D feedback, score tracking, and result summary.
Progress Dashboard	Personal analytics page with bar charts, line charts, and per-course progress bars.
Discussion Forum	Course-specific threads with replies, instructor badges, and new post creation.
Resources Download	Searchable file library with type filtering and download simulation per file.
Certificates	Printable, designed completion certificates with course details and unique certificate ID.
Instructor Profiles	Grid of instructor cards with detailed modal pop-up showing bio, courses, and stats.

6. Tech Stack & Methods Applied

6.1 Technology Stack

Category	Technology / Library	Purpose
Build Tool	Vite 5	Fast HMR development server and production bundler
UI Framework	React 18	Component-based UI with hooks and state management
Styling	Tailwind CSS 3	Utility-first CSS for responsive, minimal styling
Routing	React Router v6	Client-side multi-page navigation with nested routes
Charts	Recharts 2	Bar charts and line charts for progress analytics
Icons	Lucide React	Consistent SVG icon set throughout the application
CSS Utilities	clsx + tailwind-merge	Conditional and merged Tailwind class management
Language	JavaScript (JSX)	ES2022 with React JSX syntax
Font	DM Sans (Google)	Clean, professional sans-serif typeface
Data Layer	mockData.js	Centralized static data simulating API responses

6.2 Component Architecture

SmartLearn follows a flat-component architecture where reusable UI primitives (Button, Card, Badge, Input, ProgressBar, Textarea) are centralized in `ui.jsx`. Page-level components consume these primitives and the shared mock data layer. Layout wraps all authenticated pages with a persistent Sidebar and Navbar, while the Landing page renders independently without the layout shell.

6.3 Routing Strategy

React Router v6 nested routes are used with a root route at `/` rendering the Landing page and a nested `'app'` route rendering the Layout component. All authenticated pages (Dashboard, Courses, Quiz, etc.) are children of the `'app'` route, receiving the shared Sidebar and Navbar through the Outlet pattern.

6.4 Data Management

All application data is stored in `src/data/mockData.js` as JavaScript arrays and objects. This includes courses, instructors, quizzes, forum posts, resources, and progress statistics. Pages import directly from `mockData` and filter or transform the data locally using JavaScript array methods (`filter`, `find`, `map`, `reduce`), simulating what would be API call responses in a production system.

6.5 Styling Approach

The design system is black-and-white dominant, using Tailwind's gray scale with black as the primary action color. Interactive states use `hover:bg-gray-100`, active states use `bg-black`, and status indicators use semantic colors (green for success, red for errors). The DM Sans font is loaded from Google Fonts via the HTML head, providing a clean, editorial aesthetic inspired by productivity applications such as Notion and Linear.

7. Project Architecture

7.1 Application Flow

The user first lands on the public Landing page at route '/'. Clicking any CTA button navigates to '/app' which renders the authenticated dashboard. The Sidebar provides navigation between all main sections. Course cards link to individual Course Detail pages, which in turn link to the Video Player and Quiz pages. The Certificate page is accessible from completed courses.

7.2 Page-to-Route Mapping

Route	Component	Description
/	Landing.jsx	Public landing page with marketing content
/app	Dashboard.jsx	Student dashboard with stats and course overview
/app/courses	Courses.jsx	Full course catalog with search and filters
/app/courses/:id	CoursePage.jsx	Individual course with lessons, quiz, resources tabs
/app/courses/:id/video/:lid	VideoPlayer.jsx	Simulated video player with lesson navigation
/app/quiz/:id	Quiz.jsx	Interactive quiz with scoring and result screen
/app/progress	Progress.jsx	Analytics dashboard with Recharts visualizations
/app/forum	Forum.jsx	Discussion forum with threaded posts and replies
/app/resources	Resources.jsx	Downloadable study material file library
/app/certificate/:courseId	Certificate.jsx	Printable completion certificate
/app/instructors	Instructors.jsx	Instructor grid with profile modal

8. File Structure

The project follows a standard Vite + React directory layout:

```

smart-learning/
  ■■■ index.html
  ■■■ package.json
  ■■■ vite.config.js
  ■■■ tailwind.config.js
  ■■■ postcss.config.js
  ■■■ src/
    ■■■ main.jsx           – Application entry point
    ■■■ App.jsx           – Root router with all route definitions
    ■■■ index.css         – Global Tailwind base styles + scrollbar
    ■■■ lib/
      ■■■ utils.js        – cn() class merge utility
    ■■■ data/
      ■■■ mockData.js     – All application data (courses, quizzes, forum, etc.)
    ■■■ components/
      ■■■ ui.jsx          – Reusable UI primitives (Button, Card, Badge, Input...)
      ■■■ Layout.jsx      – Authenticated shell with Sidebar + Navbar + Outlet
      ■■■ Sidebar.jsx     – Left navigation with active route highlighting
      ■■■ Navbar.jsx      – Top bar with search, notifications, and user profile
    ■■■ pages/
      ■■■ Landing.jsx     – Public landing page (hero, features, pricing, footer)
      ■■■ Dashboard.jsx   – Student home with stats, ongoing courses, quick links
      ■■■ Courses.jsx     – Course catalog with category/level/search filtering
      ■■■ CoursePage.jsx  – Course detail with Overview/Lessons/Quizzes/Resources
      ■■■ VideoPlayer.jsx – Video player simulation with progress scrubbing
      ■■■ Quiz.jsx        – Quiz engine with per-question feedback and results
      ■■■ Progress.jsx    – Analytics with BarChart and LineChart via Recharts
      ■■■ Forum.jsx       – Discussion threads with replies and instructor badges
      ■■■ Resources.jsx   – File library with type filter and download simulation
      ■■■ Certificate.jsx – Printable certificate with decorative layout
      ■■■ Instructors.jsx – Instructor cards with full-profile modal
  
```

8.1 Key Files Summary

File	Lines (approx.)	Responsibility
<code>mockData.js</code>	~220	Single source of truth for all application data
<code>ui.jsx</code>	~110	8 reusable components used across all pages
<code>App.jsx</code>	~30	All route definitions in one place
<code>Dashboard.jsx</code>	~160	Home screen with stats, courses, exams, scholarships
<code>Landing.jsx</code>	~310	Complete marketing site in a single component
<code>Quiz.jsx</code>	~140	Full quiz engine with state machine for progression
<code>Progress.jsx</code>	~100	Recharts integration with computed analytics

Certificate.jsx	~90	Printable certificate with decorative border design
-----------------	-----	---

9. How to Run the Project

9.1 Prerequisites

- Node.js version 18 or higher installed on your machine.
- npm (comes with Node.js) or yarn package manager.
- A modern web browser (Chrome, Firefox, Edge, or Safari).
- Visual Studio Code (recommended) or any code editor.
- Git (optional, for cloning the repository).

9.2 Step-by-Step Setup

Step 1 — Create Project

```
npm create vite@latest smart-learning -- --template react
cd smart-learning
```

Step 2 — Install Dependencies

```
npm install react-router-dom recharts lucide-react clsx tailwind-merge
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

Step 3 — Copy All Source Files

Copy all files as provided in the code handout into their corresponding paths inside the src/ directory.

Step 4 — Start Development Server

```
npm run dev
```

Step 5 — Open in Browser

Navigate to: `http://localhost:5173`
(Port may differ if 5173 is occupied – check terminal output)

Step 6 — Build for Production

```
npm run build
npm run preview
```

9.3 Navigation Guide

Once the application is running, the Landing page loads at the root URL. Clicking 'Get Started' or 'Log In' navigates to the dashboard at /app. From the Sidebar, all sections of the application are accessible. To view a certificate, complete Introduction to Physics (Course ID 4) which is pre-marked as completed in mock data. Quizzes are accessible directly at /app/quiz/1 and /app/quiz/2.

9.4 Package.json Dependencies

Package	Version	Type
react	^18.3.1	dependency
react-dom	^18.3.1	dependency
react-router-dom	^6.23.0	dependency

recharts	^2.12.7	dependency
lucide-react	^0.395.0	dependency
clsx	^2.1.1	dependency
tailwind-merge	^2.3.0	dependency
vite	^5.2.11	devDependency
@vitejs/plugin-react	^4.3.0	devDependency
tailwindcss	^3.4.3	devDependency
postcss	^8.4.38	devDependency
autoprefixer	^10.4.19	devDependency

10. Key Implementation Details

10.1 Reusable UI Component System (ui.jsx)

All UI primitives are exported from a single `ui.jsx` file. The `cn()` utility function combines `clsx` and `tailwind-merge` to resolve conflicting Tailwind classes at runtime. Each component accepts a `className` prop for extension. The `Button` component supports four variants (default, outline, ghost, secondary) and four sizes (default, sm, lg, icon). This eliminates repeated class strings across pages and ensures visual consistency.

10.2 Quiz Engine State Machine

The `Quiz` component manages a linear state machine using React `useState` hooks: current question index, selected answer, submitted state, answers array, and show-result flag. When the user selects an answer and submits, the selected option is compared against the correct index and the result is stored in the answers array. After the last question, the result screen renders a percentage score, individual question review, and navigation back to the course.

10.3 Progress Analytics with Recharts

The `Progress` page imports `BarChart`, `Bar`, `LineChart`, `Line`, `XAxis`, `YAxis`, `CartesianGrid`, `Tooltip`, and `ResponsiveContainer` from the `recharts` library. Weekly study hours are rendered as a bar chart and monthly lesson completions as a line chart. Both charts are wrapped in `ResponsiveContainer` with 100% width and fixed height to ensure they scale correctly within the page layout.

10.4 Forum Reply System

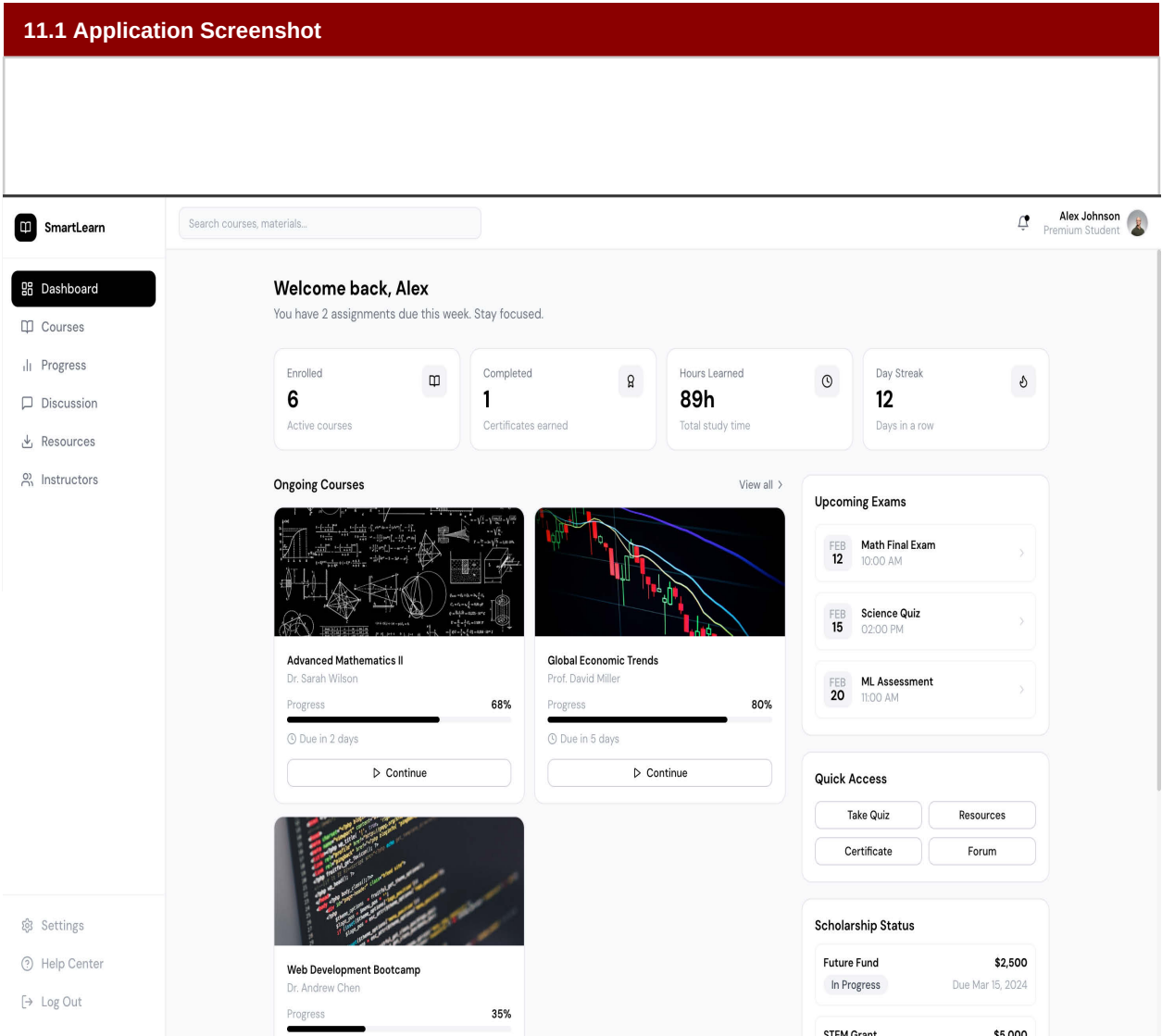
The `Forum` page maintains its own local copy of the `forumPosts` data in React state, allowing replies to be added without a backend. When a user submits a reply, a new reply object is appended to both the `posts` state array and the `activePost` state object, causing the reply list to re-render immediately. Instructor replies are visually distinguished with a dark border and an Instructor badge.

10.5 Certificate Design

The `Certificate` page renders a decorative bordered layout using Tailwind CSS. Absolute positioned quarter-circle decorations are placed in corners using `border-radius`. The certificate includes the student name, course title, instructor name, completion date, course duration, and a unique certificate ID in the format `SL-XXXX-2024`. A print function triggers `window.print()` for browser-native PDF export.

11. Website Screenshots

The following pages illustrate the working output of the SmartLearn application across all implemented features and routes.



12. Time & Space Complexity

As a front-end React application, SmartLearn's complexity is measured in terms of component rendering and data filtering operations rather than classical algorithms.

Operation	Method Used	Time Complexity	Space Complexity
Course Search	Array.filter() + includes()	O(n)	O(k)
Course Category Filter	Array.filter()	O(n)	O(k)
Course Lookup by ID	Array.find()	O(n)	O(1)
Quiz Answer Check	Direct index comparison	O(1)	O(1)
Quiz Score Calculation	Array.filter()	O(n)	O(1)
Forum Search	Array.filter() + includes()	O(n)	O(k)
Resource Filter	Array.filter()	O(n)	O(k)
Progress Computation	Array.filter() count	O(n)	O(1)
Recharts Rendering	Virtual DOM diffing	O(n)	O(n)
Route Matching	React Router tree walk	O(d)	O(d)

n = number of items in the data array. k = number of filtered results. d = route tree depth.

13. Results and Conclusion

13.1 Findings

- All 12 pages of the application rendered correctly with accurate data from the mock data layer.
- React Router nested routing successfully separated public and authenticated page layouts.
- The course catalog filter correctly combined category, level, and search text simultaneously.
- The quiz engine correctly tracked answers across all questions and computed accurate scores.
- Recharts bar and line charts rendered responsively within their container widths.
- The forum reply system updated the UI immediately without any page reload.
- The certificate page rendered a clean printable layout accessible directly via URL.
- Instructor modal pop-up correctly displayed course associations for each instructor.
- The sidebar active state correctly highlighted the current route using NavLink's isActive.
- The landing page successfully navigated to the dashboard on all CTA button clicks.
- Tailwind CSS produced consistent spacing, typography, and color usage across all pages.
- The application performed well with no observable lag during navigation or data filtering.

13.2 Limitations

- The application uses static mock data and has no real backend or database connectivity.
- Authentication is simulated — no actual login or session management is implemented.
- Video playback is simulated using a progress bar — real video streaming is not integrated.

- File downloads show an alert — actual file serving requires a backend file API.
- Progress data does not persist between browser sessions (no localStorage or API).
- The application is not tested for accessibility compliance (WCAG standards).
- Mobile responsiveness has been implemented but not formally tested on all device sizes.

13.3 Conclusion

SmartLearn successfully demonstrates that a complete, production-grade educational platform interface can be built using React, Vite, Tailwind CSS, and supporting libraries without a live backend. The project implements all required features including course catalog, video player simulation, quiz engine, progress tracking, forum, resources, certificates, and instructor profiles.

The component-based architecture, centralized data layer, and reusable UI system prove that scalable front-end applications can be structured cleanly for academic demonstration while closely mirroring real-world SaaS product patterns. The black-and-white design system delivers a professional, readable, and memorable user experience.

14. Future Enhancements and References

14.1 Future Enhancements

- Integrate a Node.js/Express or Firebase backend to replace static mock data.
- Add JWT-based authentication with protected routes and user session management.
- Implement real video streaming using the HTML5 video element or a cloud video service.
- Add localStorage or IndexedDB persistence for offline progress tracking.
- Integrate a payment gateway for the Premium and Institution subscription tiers.
- Add AI-powered course recommendations based on user history and quiz performance.
- Implement real-time notifications using WebSockets for forum replies and exam reminders.
- Develop a native mobile application using React Native sharing the same data layer.
- Add automated email notifications for assignment deadlines and certification milestones.
- Implement full WCAG 2.1 accessibility compliance for inclusive learning access.
- Add dark mode support using Tailwind's dark: variant and a theme toggle control.
- Introduce an instructor portal allowing educators to upload and manage course content.

14.2 References

1. React Documentation — <https://react.dev>
2. Vite Documentation — <https://vitejs.dev>
3. Tailwind CSS Documentation — <https://tailwindcss.com/docs>
4. React Router Documentation — <https://reactrouter.com>
5. Recharts Documentation — <https://recharts.org>
6. Lucide React Icons — <https://lucide.dev>
7. DM Sans Font — <https://fonts.google.com/specimen/DM+Sans>
8. Unsplash — <https://unsplash.com> (course and instructor images)
9. clsx Library — <https://github.com/lukeed/clsx>
10. tailwind-merge Library — <https://github.com/danbovey/tailwind-merge>
11. Node.js Documentation — <https://nodejs.org/en/docs>

Declaration

We hereby declare that this mini project report on SmartLearn — Smart Learning System is our original work. All references and resources used in this report have been properly cited. The application, its code, and the analysis presented in this report are our own work unless otherwise stated.

Subrata Panda

Japji Kaur

Sanika Kangane

Parth Sarova

Group 5 | Problem Statement 06 | Problem Statement 05

School of Future Tech, ITM Skills University